also like to be able to respond to these options by running code in your script based on them. Ruby provides the conveniently named class OptionParser to allow you to run code based on command-line options, or to convert those options into a more convenient data object.

The API here is a little tricky, but the basic idea is:

- Create a new instance of OptionParser
- Tell the instance about various options it should respond to and what it should do for each one
- Tell the instance to parse, and also to do something with the options as it parses.

Along the way, the OptionParser instance also does a few things for you, including automatically generating a --help response based on the options. We'll also note here that the OptionParser class has a couple of different ways to do the things we are discussing in this section, we're only going to talk about the main one but the official API reference has other options.

To create the parser, you can just use parser = OptionParser.new. Then you can define options to match with the method on. For option parser purposes, an option is either:

- short: a single dash and a single character, as in -x
- long: a double dash and more than one characters, as in --database.

The basic idea is the same either way, the on method is called with the option, an optional string that defines it, and a block that is invoked if the option is in ARGV. You can also define both a short and a long version of the same option.

rubyworld/option_1.rb

```
require "optparse"

parser = OptionParser.new

sort_type = nil
parser.on("-a", "Alphabetical") do
  sort_type = :alphabetical
end

parser.on("--recent", "Most Recent") do
  sort_type = :recency
end

parser.on("-s", "--size", "Size") do
  sort_type = :size
end

parser.parse!

p "we are sorting by #{sort_type}"
```

In this example, each option is setting a sort type local variable. Options created with on are invoked in the order they are defined, so if more than one of these is invoked, last one defined wins. The parse! at the end is what actually triggers the parsing of the options, it's also a signal that you are done defining options.

At this point, if you call your script with a --help option (whether you do it via ruby or if you make it a standalone executable script), you get a useful help message:

```
$ ruby code/rubyworld/option_1.rb --help
Usage: option_1 [options]
    -a                               Alphabetical
```